Kasparro AI Engineer assignment

Fresher AI Engineer | AI visibility analysis and human review

Pack v1.0 | 27 October 2026 | Six active hours | Five calendar days by default

Your challenge

Build a small runnable backend that turns 50 prompts and 150 saved answers from ChatGPT, Gemini and Google AI Mode into transparent scores, three evidence-backed diagnostics and three prioritised remedies. A human must be able to edit and review a report, and the backend must control which version can be exported.

Your product should answer: where does Kasparro appear to be winning or losing in this snapshot, what evidence supports that finding, and what should be investigated or changed next?

Brand and domain context

Kasparro helps brands assess their representation in AI answers. Its public site describes audits covering search visibility, trust signals, content quality and transaction readiness. This assignment studies Kasparro itself as a provider, using real archived answers. Public demo figures are illustrative; company claims require supporting evidence.

SEO means search engine optimisation. AEO and GEO refer here to work on accurate representation in answer engines and generative AI. A prompt is the question submitted; a surface is the particular AI experience. Inclusion means appearing in an answer. Recommendation means being presented as a suitable option. A citation is a returned link, whose support for a claim still needs inspection.

Working conditions

- Stop at six active hours. Record time spent, deliberate omissions and unfinished work. Aim for a correct complete basic flow and defensible decisions. Reading, coding, analysis, evaluation and documentation count as active work.
- Use Python preferably, or another language you can defend. SQLite is sufficient. A dashboard, Docker, public deployment, crawling, vector search and multiple agents are optional and earn no separate feature bonus.
- Use an LLM for at least one substantive step, such as semantic classification or diagnostic synthesis. Use deterministic code for exact counts, validation, aggregation and permissions. AI coding assistants are welcome; disclose and verify your contribution.

The follow-up includes your code, results and a small live change. Extra features do not offset broken core behaviour.

Inputs and how to start

The baseline is one historical run captured on 31 July 2026 in India and English through DataForSEO. It is a snapshot of those experiences, not a current market ranking or a direct call to each vendor API. All 50 prompts have one captured response from each engine.

File	What to use
data/prompts.jsonl	50 exact questions and stable prompt IDs.
data/responses.jsonl	150 full archived answers, markdown, engine/model metadata and capture details.
data/citations.jsonl	2,439 returned citation occurrences linked by response ID.
data/run.json and data/READ_ME.txt	Run provenance, limits and field definitions.
reference/	Dated brand schema and short official domain references.
provenance/provider_payloads.jsonl	150 original archived provider JSON payloads for traceability.
schemas/ and fixtures/	Input record schema and controlled failure scenarios.
tools/check_pack.py	Offline input-integrity checker; it does not grade your solution.

Unzip this pack and read data/READ_ME.txt. With Python 3.10 or later, run python3 tools/check_pack.py from the extracted pack directory. On Windows, py -3 tools/check_pack.py is equivalent. The checker uses only the Python standard library. You may use another language to read the JSON files.

Evidence rules

- Preserve original inputs and IDs, then map them to your own validated contract. Do not invent missing metadata. In particular, unknown model versions and citation publication dates remain unknown.
- Treat answer_text and answer_markdown as two views of one response. A URL can recur inline, in a sources panel and in another answer. Define your counting and deduplication rules.
- Infer useful prompt categories from the actual questions. Existing production labels and scores are not supplied. Citation presence is observable; source support remains unverified unless you inspect the relevant material.
- Live website reading and optional AI checks can support orientation. Keep new observations separate, dated and clearly labelled. Fresh collection and paid accounts are not prerequisites.

Use a local model or hosted access explicitly provided or approved by the hiring team. You are not expected to buy credits, a subscription or a deployment. Variable paid usage must stay within US$5 if approved credits are available. Record model/configuration, calls, time and available cost measurements.

A keyless replay or stub mode is permitted when model access is unavailable. Build the real model adapter and prompts, and document the command to switch to it. Keep the full workflow or application runnable in replay mode. Label every output and trace with its execution mode. Replay supports mechanics and failure testing; it does not establish live-model correctness, variability, cost or latency.

Build the analysis service

Classification and scoring

Account for every one of the 150 responses with a structured result, explicit failure or skipped reason. At minimum, distinguish Kasparro absent, mentioned, recommended and negatively characterised; labels may overlap where justified. Extract explicitly named competitors, recommendation position when observable, and supplied citations.

Propose a transparent visibility score. Explain its scale, components, weights and uncertainty handling. Preserve component values and exact evidence excerpts or locations. A mention is not automatically a recommendation, and a failed analysis is not a successful zero-score observation.

Backend capabilities

Persist raw inputs, analysis results, report versions and review decisions. Keep sensible boundaries between ingestion, model access, scoring, storage and APIs. Include a health endpoint and generated API documentation or example requests. Paths below are illustrative; equivalent contracts are acceptable.

Capability	Example contract
Import and validate inputs	POST /datasets
Run analysis and inspect status	POST /analyses; GET /analyses/{id}
Edit a draft or its findings	PATCH /reports/{id}
Submit an immutable version	POST /reports/{id}/submit
Approve or reject that version	POST /reports/{id}/decision
Export an approved version	GET /reports/{id}/export

Execution and data integrity

- Validate requests and model outputs. Bound timeouts, retries, calls and output sizes. Handle empty or malformed output and provider failure without manufacturing a confident result.
- Trace response IDs through processing status, prompt/model/scoring configuration and evidence. Distinguish live, local-model, replay and stub calls. Keep secrets in environment variables and provide .env.example.
- Treat source text and prompts as untrusted data. They cannot change tool authority, reviewer identity or export permission. One bad record must not erase valid results.
- Use appropriate HTTP status codes and useful structured errors. A bounded synchronous flow is acceptable; explain its limits. Dataset, Response, AnalysisRun, ResponseResult, ReportVersion and ReviewDecision are suggested entities, not required class names.

A repeated import of the same identified response must be rejected or safely reused, with no silent duplication. Explain the approach. Full restart/resume and simultaneous-review protection are follow-up discussion topics, not required implementations.

Human review and evaluation

Review must control export

Configure two local identities or tokens: analyst and reviewer. The analyst can import, analyse and edit drafts; the reviewer can approve or reject a submitted report version with a comment. Derive identity and permissions from backend configuration. A caller-supplied role or actor name is not authorisation. Full OAuth is unnecessary.

Submitting freezes the version under review. Present its findings, evidence and remedies. Persist draft, submitted, approved and rejected states, with actor, decision, version, comment and time. Export must refuse unapproved, rejected and stale content. Editing a submitted or approved report creates a new draft and requires a new review. An older approval may authorise export only of that clearly identified immutable older version.

Demonstrate wrong-role denial, rejection blocking export, and edit-after-approval requiring a new review. Describe an expiry policy and concurrency limits as future work; implementing them is optional.

Create and run your evaluation

- Before tuning, manually label at least 8 supplied responses spanning all three engines and several prompt categories. Include varied presence, recommendation, negative characterisation and citation cases where available. Add 2 clearly marked edge cases of your own. State selection and keep tuning cases separate from the final check.
- Store versioned expected labels/components, source evidence and a runnable grader. Use per-label precision/recall or agreement measures, and separately verify deterministic score calculations. Report eligible, successful, failed and skipped counts; failures must remain visible in end-to-end results.
- Check whether diagnostic claims have supporting response evidence and whether remedies follow from them. Include an unsupported-claim or incorrect-evidence case. Explain grader limits; an LLM judging its own prose is insufficient.
- Compare a simple rule-based baseline with your approach on the same cases. Repeated-model studies are optional. If only one real-model run is available, state that limitation.
- Show one failure, your diagnosis, a change and a regression case. If necessary, create and label a controlled edge case. Supply one command that reruns the evaluation and writes a readable comparison report.

In replay-only mode, you must still implement and demonstrate the application, permissions, evaluation runner and failure paths. Mark model-quality results unverified, and give the reviewer the command/configuration needed to run real-model cases.

Findings submission and assessment

Diagnose and prioritise

Compare engines and prompt categories with clear denominators, coverage and error counts. Compare the same prompts where possible, and identify scoring and category versions. Investigate the three most important wins or losses, linking each to response IDs and excerpts. Separate the recorded pattern, a possible explanation and evidence that would test it.

Propose three prioritised remedies. For each state the problem, supporting evidence, action or investigation, expected benefit, effort, uncertainty and a measurable validation step. Do not promise ranking improvements or invent Kasparro facts. An investigation can be the best next action when evidence is inadequate.

Required backend checks

Include automated tests for invalid input, duplicate response handling, model timeout or malformed result, incorrect reviewer role, rejection blocking export and edit-after-approval. Deterministic model stubs are acceptable for these engineering tests; distinguish them from model-quality evaluation.

What to submit

- Private source archive or repository: code, database schema/initialisation, dependency versions, .env.example, tests, evaluation cases/runner/results and a controlled fixture. Keep supplied data and private outputs out of public Git history.
- README with exact setup, database initialisation, start/test/evaluation commands and example requests for analyse > edit > submit > approve/reject > export. Include keyless replay mode and a real-model adapter. Target under ten minutes after prerequisites; disclose model-download delays.
- Analysis output: scores and coverage, three diagnostics, three remedies, and actual active time. Plain JSON is sufficient. Include a decision note of about one page: scoring/aggregation, backend and review/reliability choices, alternatives, top remedy, failure/fix, largest risk, omissions, questions before release and AI assistance.

Assessment

Area	Weight
Backend engineering	25%
Execution harness	15%
Evaluation harness	20%
Human review enforcement	15%
Judgment and ownership	20%
Reproducibility	5%

We assess a correct complete basic flow, readable code, defensible reasoning and learning from a failure. Advanced infrastructure and visual polish are not required.

Submit within five calendar days of receiving the complete pack, unless the invitation gives another calendar deadline. The active-work limit still applies. Reply to the email that supplied this pack with a private archive or private repository and any requested recordings. Grant access to that sender and any reviewers named in the invitation. If links cannot be shared, attach an archive. Report missing files or access blockers in the same email; do not buy access to resolve them.

This is a simulated client-report export. Do not contact customers, publish findings, execute remedies or change a real website. Stop at six active hours and disclose unfinished work.